

Course Project

Design & Implementation of a Mini Programming Language

A language-design project covering Chapters 1–7 of Sebesta's *Concepts of Programming Languages* (12th ed.)

Team size	Solo or pair (pair declaration form due 4 May 2026)
Part 1 submission	Friday, 8 May 2026, 23:59
Part 1 written exam	Thursday, 14 May 2026, 08:30 — 90 min, in class
Part 2 submission	Friday, 22 May 2026, 23:59
Part 2 written exam	Thursday, 28 May 2026, 08:30 — 90 min, in class
AI tool use	Mandatory, must be documented (see §6)

1. Project Overview

In this project you will **design your own small programming language** (a **domain-specific language**, or DSL) and implement an **interpreter** for it. The project is intentionally structured so that no AI tool can complete it end-to-end: the core deliverable is **your design decisions** and your ability to **defend them** using the vocabulary of Sebesta's textbook.

You are **required** to use AI assistants (Claude, ChatGPT, Copilot, Gemini, etc.) during the project, and you must document that use in an **AI Usage Journal**. The goal is not to prevent AI-assisted work but to train you to use AI *as an engineer does* — critically, with judgment, and with full understanding of the output.

The project is delivered in **two parts**: Part 1 (foundations: design, lexer, parser) on **8 May**, and Part 2 (complete language: type checker and interpreter, with all of Part 1 revisable) on **22 May**. Each part is graded independently and is followed by an in-class **written exam** with questions personalised to your submission.

The one rule that makes this project AI-resistant

Each part is followed by a **90-minute written exam** in class with **questions tailored to your own submission**. Some questions are project-specific (“In your D1 you said your language has static scoping — show on this code excerpt how the lookup of variable x resolves.”). Some are general PL questions on the corresponding Sebesta chapters. If you cannot answer questions about your *own* design and code, your grade will reflect that — regardless of how polished the written submission looks. The exams carry substantial weight in the final grade.

2. Choosing Your Language Domain

Each student/pair must choose a **distinct, non-trivial application domain**. You are strongly encouraged to pick a domain that is different from your classmates' — suspiciously similar projects will be investigated for plagiarism (see §9), and originality is one of the graded rubric dimensions (see §5).

Acceptable examples

- **TurtleLang** — a language for drawing geometric shapes with turtle graphics, with recursion and user-defined shape procedures.

- **ChessDSL** — a language for describing and solving chess puzzles (piece placement, legal moves, constraint expressions).
- **QueryLite** — a mini-query language over CSV files, with filters, joins, and aggregates.
- **MusicScript** — a language for describing melodies (notes, durations, loops, transposition).
- **CircuitLang** — a language for describing digital circuits (gates, wires, simulation).
- **RecipeLang** — a language for scaling and substituting ingredients in recipes, with unit types.
- **PlotDSL** — a language for declarative 2D data plots with statistical transforms.

Minimum requirements for any chosen domain

- At least **three primitive types** and **one structured type** (array, record, or user-defined).
- At least **one control structure** beyond sequencing (if, while, for, or pattern-match).
- **User-defined functions or procedures** with parameters.
- At least **one domain-specific construct** that justifies why this is a *DSL* and not a generic scripting language.
- A nontrivial expression language with operators that raise interesting design questions (precedence, associativity, mixed-mode arithmetic, coercion).

3. Structure of the Project

The project has **two parts**, each with a written submission and an in-class written exam. Both parts can be done **solo or in pairs**. The detailed specifications of each deliverable are in §4.

3.1 Working in Pairs

You may work alone or in pairs of two. Pairs choose their own partner.

- **Pair declaration deadline:** Monday, **4 May 2026**. A form will be distributed; complete it once and submit by that date. Students who do not submit the form by 4 May are treated as solo.
- **Even split required.** One partner must **not** do most of the work. A reasonable split is by component (e.g., one writes the lexer + parser, the other writes the type checker + interpreter), or by features within each component. Lopsided contributions will be detected through the exams (see §3.3) and graded down.
- **Each student writes their own AI Usage Journal** documenting their own AI interactions. The journal is individual, not shared, even within a pair.
- **Each student takes their own written exam** with questions targeted at the components they personally worked on.
- **Solo students submit no contribution report** — pairs must (see D6 below).

3.2 Deliverables, by Part

Each part is graded as a **separate project**. Part 2 is a re-submission of the entire project — you may freely revise everything from Part 1 in light of feedback received. The two parts are weighted equally.

#	Deliverable	Part 1 — due 8 May	Part 2 — due 22 May
D1	Design Specification	Initial version covering Sebesta Ch. 1–5 sections only (language overview, lexical structure, syntax in EBNF, scope/binding/lifetime).	Complete final version covering all of Ch. 1–7 (adds the type system, expressions section, semantics for two constructs, and full design rationale). All Part 1 sections may be revised.

#	Deliverable	Part 1 — due 8 May	Part 2 — due 22 May
D2	Implementation	Lexer + parser. Accepts valid programs and produces a printable AST; rejects malformed programs with a useful error message including line number.	Adds type checker and interpreter. Full end-to-end execution of the three example programs of D3.
D3	Example Programs & Test Report	Three non-trivial programs that parse successfully, plus five malformed programs showing parser error messages. PDF, 2–3 pages.	Same three programs, now executed with their actual outputs. Adds at least one program that triggers a type error caught before execution. PDF, 3–5 pages.
D4	AI Usage Journal (<i>per student</i>)	Entries dated up to 8 May. Minimum 4 substantive entries. Includes Experiment E1 (see §6).	Continuation: entries dated 9–22 May. Minimum 6 additional entries. Includes Experiments E2 and E3 (see §6).
D5	Retrospective & Self-Assessment	Half a page: what is working, what is hard, what you plan to fix in Part 2.	Full one-page retrospective and self-assessment against the rubric in §5.
D6	Contribution Report (<i>pairs only</i>)	For each Part 1 deliverable section, name the partner who owned it and the partner who reviewed it. Half a page.	Updated report covering the full project , listing every section/component and the owning partner. Half a page.

Late policy: –10% per day late, up to 3 days; zero afterwards. Applied to each part separately. Extensions require documented medical or personal justification and must be requested **before** the relevant deadline.

3.3 In-Class Written Exams

Each part is followed by a **90-minute written exam** in our regular lecture room and time slot. Each student receives a **unique exam paper** with questions tailored to their own submission.

	Exam 1	Exam 2
When	Thursday, 14 May 2026 , 08:30	Thursday, 28 May 2026 , 08:30
Duration	90 minutes	90 minutes
Covers	Your Part 1 submission and the corresponding Sebesta chapters (Ch. 1, 3, 4, 5).	Your full Part 2 submission and the corresponding Sebesta chapters (Ch. 6, 7), with possible reference back to Part 1 material.
Format	Written, on paper. Same lined-and-bordered answer-box style as the midterm. Some short-answer questions, some require you to draw or annotate.	Same.

	Exam 1	Exam 2
Questions	Mostly project-specific (about your own design, code, and choices). Some general PL questions on the corresponding chapters.	Same.

What you may bring to the exam

Three sheets of A4 of handwritten notes, one side each. No printed or photocopied material — your notes must be handwritten by you. Each sheet must carry your **name and student number** at the top. All notes are collected at the end of the exam. You may not consult laptops, phones, books, or any printed material during the exam.

Why the two-part structure does not make this an “all-nighter project”

The two parts depend on each other. Your Part 2 implementation must be consistent with the spec you submitted in Part 1 (you can revise the spec — but not retroactively). Your AI Usage Journal entries must be dated across the working period of each part, not backfilled. And the in-class exams will surface any mismatch between what you wrote and what you understand. Start early and work steadily.

4. Deliverable Specifications

D1 — Design Specification Document

A PDF report, ultimately 8–12 pages, containing the sections below in this order. Sections marked **[P1]** are required for the Part 1 submission; sections marked **[P2]** are added in Part 2. All [P1] sections may be revised between submissions.

4.1 Language Overview **[P1]** (½ page)

Name of your language, intended domain, sample program, and a one-paragraph justification citing Sebesta's **language evaluation criteria** (readability, writability, reliability, cost — Ch. 1). Which criteria does your DSL prioritize, and which does it knowingly sacrifice?

4.2 Lexical Structure **[P1]** (½ page)

List of **token categories**: identifiers, literals, keywords, operators, separators. Give the regular expression or pattern for each.

4.3 Syntax **[P1]** (2 pages)

Complete grammar in **EBNF** (Sebesta §3.1–3.2). Must be unambiguous for the subset you implement. Annotate any places where you resolve ambiguity (e.g., operator precedence, dangling else).

```
/* Example EBNF fragment, Sebesta style */
<expression> ::= <term> { ("+" | "-") <term> }
<term>       ::= <factor> { ("*" | "/" ) <factor> }
<factor>     ::= <ident> | <number> | "(" <expression> ")"
```

4.4 Semantics **[P2]** (2 pages)

Give either **operational** or **denotational** semantics (Sebesta §3.5) for *at least two* non-trivial constructs from your language — e.g., the while loop or your domain-specific construct. Choose one style and apply it consistently. Axiomatic semantics is *not* accepted for this project.

4.5 Type System **[P2]** (1–2 pages)

Answer all of the following in prose, each answer tied to Sebesta Ch. 6:

- Which **primitive types** does your language have, and what are their value sets?
- Is your language **strongly typed** (§6.12)? Justify.
- Does your language allow **implicit type coercion** (§7.4)? If yes, state the coercion rules.
- What is your rule for **type equivalence** — name equivalence or structural equivalence (§6.14)? Why did you choose it?
- For your structured type (array/record/...), give the design decisions from Sebesta §6.5–6.7 that apply (e.g., for arrays: static vs. dynamic, index range, subscript check).

4.6 Names, Binding, Scope, Lifetime [P1] (1 page)

Answer, citing Sebesta Ch. 5:

- What are the rules for **legal identifiers**?
- What **bindings** happen at compile time vs. run time?
- Does your language use **static scoping** or **dynamic scoping**? Why? What would break if you switched?
- What is the **lifetime** of local variables — stack-dynamic, static, or explicit-heap-dynamic?

4.7 Expressions & Assignment [P2] (½ page)

State your decisions from Sebesta Ch. 7:

- **Operator precedence** and **associativity** table.
- Do you evaluate Boolean operators with **short-circuit** semantics?
- Order of operand evaluation — defined or unspecified?
- Assignment: is it a **statement** or an **expression**?

4.8 Design Rationale [P2] (1 page)

For each of the five numbered design decisions from sections 4.5–4.7 above, write one paragraph justifying your choice and naming at least one **trade-off** you accepted. This section is where AI-generated boilerplate fails most visibly; write in your own voice.

D2 — Implementation

Submit source code (zip or public Git repo link). The required scope grows from Part 1 to Part 2.

[P1] — due 8 May:

- A README with build/run instructions and the command line used.
- A working **lexer** and **parser** that **accept** all valid programs in your D3 test suite.
- **Reject** malformed programs with a useful error message including line number.
- Produce a printable AST for each accepted program (a `--dump-ast` flag or equivalent).

[P2] — due 22 May, in addition:

- A **type checker** consistent with the type system you declared in your final D1 (strong/weak typing, coercion rules, type equivalence).
- An **interpreter** that executes programs respecting your declared scoping rule, precedence, associativity, and parameter-passing mode.
- Informative **runtime errors** for division by zero, index out of range (if applicable), and at least one domain-specific error.
- End-to-end execution of all three example programs from D3 with the expected outputs.

Implementation language is up to you: Python, JavaScript/TypeScript, C, C++, Java, C#, Go, Rust, OCaml, Haskell. Parser generators (ANTLR, yacc/bison, pegjs) are allowed as long as you can explain them in your exam.

Consistency between D1 and D2 is graded strictly

Your implementation must match the language you described in D1. If your D1 says “static scoping” but your interpreter resolves variables dynamically, **both** D1 and D2 lose points, because it means you did not understand your own design. You are allowed to revise D1 — as long as the final submitted spec matches your final submitted implementation.

D3 — Example Programs & Test Report

[P1] — **due 8 May**: a short PDF (2–3 pages) containing:

- **Three non-trivial example programs** in your language that together exercise every control structure, your structured type, function definition and call, and your domain-specific construct. They must **parse** successfully — they need not yet execute.
- **Five malformed programs** that demonstrate parser error messages. For each: the source, the error message your parser produces, and the line number.

[P2] — **due 22 May**: updated PDF (3–5 pages) adding:

- For each of the three example programs: the **actual output** when run under your interpreter, plus a brief (3–5 sentence) discussion of what it demonstrates about your language.
- **At least one program that exercises your type checker** — i.e., a type error that is caught before execution. Show the type error message.

D4 — AI Usage Journal (*per student*)

See §6 for full specification, format requirements, and the entry template. The journal is a separate PDF (one per student, even within pairs) with timestamped entries across the working period of each part. Backfilled or undated journals receive zero. The journal is split: at least 4 entries by 8 May, at least 6 more by 22 May.

D5 — Retrospective & Self-Assessment

[P1] — **due 8 May**: half a page — what is working, what is hard, what you plan to fix in Part 2.

[P2] — **due 22 May**: one full page with two sections:

- **Retrospective (½ page)**: What would you change if you were to start over? What did not work and had to be scoped out of your implementation? What was the single hardest design decision, and why?
- **Self-Assessment (½ page)**: Using the rubric in §5, grade yourself on each of the six dimensions (A / B / C / D). Give a one-sentence justification for each grade. A thoughtful, honest self-assessment — including identifying your own weaknesses — receives full credit; a reflexive “A across the board” does not.

D6 — Contribution Report (*pairs only — solo students skip*)

Half a page, signed by both partners. Required at **both** Part 1 and Part 2 submissions.

- List **every section of D1, every component of D2, every program in D3, and the section of D5** covered by the current submission. For each item, name the partner who **wrote it** and the partner who **reviewed it**.
- State explicitly that the workload is split evenly. If it is not, explain why and propose how Part 2 will rebalance.
- Each partner's contribution should be at least 40% of the total work measured roughly. Lopsided splits are detected through the exams (each partner gets questions about **their own** components) and graded down for the partner who did less.

5. Grading Rubric

Each part is graded on all six dimensions below: the written artifacts contribute to the first five dimensions, and the in-class written exam contributes to the sixth. The two parts are weighted equally in the final project grade.

Dimension	Excellent (A)	Good (B)	Adequate (C)	Weak (D/F)
Correctness	All stated behaviors work; no contradictions between spec and implementation.	Minor bugs; ≤ 2 contradictions that are clearly acknowledged.	Core features work; edge cases fail; some inconsistencies with spec.	Spec and implementation diverge significantly; major features missing.
Design Justification	Every major decision justified with Sebesta terminology and a named trade-off.	Most decisions justified; terminology mostly correct.	Generic justifications; terminology used loosely.	Decisions stated without justification, or justifications are boilerplate.
Use of Sebesta Framework	Accurate use of grammar formalism, semantics style, type-system and binding terminology.	Accurate terminology with 1–2 misapplications.	Terminology mostly present but shallowly applied.	Terminology misused or absent.
Originality	Clearly distinct DSL with a genuine domain-specific construct.	Distinct DSL; domain-specific construct is present but thin.	DSL is a lightly themed reskin of a generic language.	Language is indistinguishable from a generic textbook toy language.
Code Quality	Clean architecture; lexer, parser, type checker, and evaluator clearly separated.	Readable code with a working separation of concerns.	Code works but is tangled; separation of concerns unclear.	Code is opaque, undocumented, or does not match the spec.
Exam Performance	Confidently answers project-specific and general PL questions; can explain alternatives not taken.	Answers most questions correctly; recovers from follow-ups with light prompting.	Answers surface questions; cannot discuss design alternatives or apply concepts to own code.	Cannot answer fundamental questions about own submission or the corresponding chapters.

How the exam interacts with the rest of your grade

If your exam answers demonstrate that you do **not** understand a section of your own submission at a basic level — for instance, you cannot explain what *static scoping* means in your own interpreter — the instructor reserves the right to reduce the corresponding artifact's grade accordingly. Conversely, strong exam answers can recover points on written work that had unclear justification.

6. The AI Usage Journal

You are required to use AI assistants on this project. It is also a graded deliverable that you document that use. The rationale is professional: modern software engineers work with AI tools as a matter of course, and a course on programming languages is a natural place to practice using them with judgment rather than blind acceptance.

Format

Each student writes their own journal — even within a pair, the journal is individual. Submitted as a PDF; the journal is itself split between the two parts.

- **Part 1 journal** (submitted 8 May): minimum 4 substantive entries, dated up to 8 May. Must include Experiment **E1** below.
- **Part 2 journal** (submitted 22 May): minimum 6 additional entries, dated 9–22 May. Must include Experiments **E2** and **E3** below.
- Each entry is a discrete **session** with an AI tool. You do not need to log every trivial autocomplete — aim for the 10+ most substantive interactions per student across the whole project.
- **Every entry must carry a real date.** A journal whose entries are all dated in the final 48 hours is not credible and will receive zero.

Required contents of each entry

- **Date** and which AI tool/model you used.
- **What you were trying to do** (one sentence).
- **The prompt you sent**, verbatim.
- **The relevant portion of the response** (verbatim excerpts or a tight summary).
- **What you accepted, what you rejected, and why** — this is the graded part.
- **Any errors or misleading claims the AI made**, and how you detected them.

Three mandatory experiments

At least three of your journal entries must document the following experiments:

#	Experiment	What to report
E1	Ask the AI to generate an EBNF grammar for your language from a one-paragraph description.	Compare its grammar with yours. What did it get wrong or suboptimal? Does it handle operator precedence correctly?
E2	Ask the AI to explain the difference between name equivalence and structural equivalence for your record type.	Does its explanation match Sebesta §6.14? Does the example it gives actually run under the equivalence rule it described?
E3	Ask the AI to implement your type checker (or a non-trivial portion of it).	Run its code on at least three test inputs where you know the right answer. Report any bugs, subtle or blatant, and how you fixed them.

How the journal is graded

- **Honesty.** Fabricated or copy-pasted-for-show entries are worse than none and will be treated as academic dishonesty.

- **Depth of reflection.** Entries that merely say “AI gave correct answer, I used it” get minimum credit. Entries that discuss why the answer was correct, what could have gone wrong, or how you verified it, get full credit.
- **Critical evaluation.** The journal should contain at least three documented cases where the AI was wrong, unclear, or suboptimal.

What is not allowed

You may not paste AI-generated text directly into your Design Specification, Final Report, or AI Usage Journal reflections without marking it as a quotation and discussing it. The *rationale* sections and the *reflection* parts of the journal must be in your own voice. AI-assisted **code** is fine — but you must be able to explain every line you submit.

7. Sample Language Specification: “ShapeScript”

What follows is a **deliberately short example** of the kind of spec we expect — *shorter* than yours will be, so you can see the shape of it without being tempted to copy its design choices. Your spec must make **different** choices from this one in at least three places. (Otherwise your domain probably isn't different enough.)

7.1 Overview

ShapeScript is a DSL for procedurally describing 2D geometric compositions. It prioritizes **writability** (Sebesta §1.3.2) for domain users and **reliability** via strong typing over raw numeric performance, which it knowingly sacrifices.

```
shape flower(petals: int) {
  repeat petals times {
    draw circle(radius: 10)
    rotate 360 / petals degrees
  }
}
draw flower(petals: 8)
```

7.2 Lexical Structure (excerpt)

```
IDENT      = [a-zA-Z_][a-zA-Z0-9_]*
INT_LIT    = [0-9]+
FLOAT_LIT  = [0-9]+ "." [0-9]+
KEYWORDS   = shape | draw | repeat | times | rotate | degrees | if | else
OPERATORS  = + - * / == != < <= > >=
SEPARATORS = ( ) { } , :
```

7.3 Syntax (excerpt, EBNF)

```
<program>    ::= { <shape_decl> | <stmt> }
<shape_decl> ::= "shape" IDENT "(" [ <params> ] ")" <block>
<params>     ::= IDENT ":" <type> { "," IDENT ":" <type> }
<stmt>       ::= <draw_stmt> | <repeat_stmt> | <rotate_stmt> | <if_stmt>
<repeat_stmt> ::= "repeat" <expr> "times" <block>
<expr>       ::= <term> { ("+" | "-") <term> }
<term>       ::= <factor> { ("*" | "/" ) <factor> }
```

7.4 Type System

- **Primitive types:** int, float, bool.
- **Structured type:** point, a record with fields x: float and y: float.

- **Strong typing:** yes. Every operator is defined only on matching operand types.
- **Coercion:** int is implicitly widened to float in mixed-mode arithmetic; float is **not** narrowed to int.
- **Type equivalence:** name equivalence for records — two record types are equivalent only if they have the same declared name.

7.5 Scope, Binding, Lifetime

- **Scoping:** static (lexical). Each shape body opens a new scope.
- **Bindings:** variable types are bound at declaration time. Shape definitions are bound before the first draw.
- **Lifetime:** parameters and locals are stack-dynamic; shape declarations have static lifetime (whole program).

7.6 Expressions

- **Precedence:** unary minus $> * / > + - >$ comparison $> \&\& > ||$.
- **Associativity:** all binary operators are left-associative.
- **Short-circuit:** $\&\&$ and $||$ short-circuit left-to-right.
- **Operand evaluation order:** left-to-right and fully defined, for reliability.
- **Assignment:** a statement, not an expression — no assignment-in-expression surprises.

7.7 Semantics (operational, sketch)

Let S be the state (current turtle position, heading, canvas). The semantics of repeat n times B is, in big-step operational form:

```

<n, S> -> n'      <B, S> -> S1
<B, S1> -> S2    ... <B, S_{n'-1}> -> S_{n'}
-----
<repeat n times B, S> -> S_{n'}

```

Note how this reveals a design decision: n is evaluated **once**, not each iteration. Changing this would change the semantics; the design rationale must explain the choice.

7.8 Design Rationale (excerpt)

Why name equivalence for records? Structural equivalence would conveniently let users pass any $\{x, y\}$ -shaped record to a function expecting point, but it would also silently accept any accidental $\{x, y\}$ -shaped record that was intended to represent a *vector* or a *dimension*. For a DSL aimed at correctness in a geometric setting, name equivalence catches this confusion at type-check time. The trade-off accepted: more point constructor calls in user code.

8. AI Usage Journal — Entry Template

Copy this template for every journal entry. Entries should be concise ($\frac{1}{2}$ to 1 page each). The minimum entry counts (4 by 8 May, 6 more by 22 May) are stated in §6.

Field	Content
Entry #	e.g., 7 of 14
Date	YYYY-MM-DD (must be a real date during project work)
Phase	Design / Lexer / Parser / Type checker / Interpreter / Testing / Writing
AI tool	Claude Opus 4.7 / ChatGPT / Copilot / Gemini / other (+ version if known)

Field	Content
Goal	One sentence — what you were trying to achieve. <i>Example: “Write the pretty-printer for my AST.”</i>
Prompt	Verbatim. If you iterated, show the final prompt plus a one-line summary of what changed.
Response (key part)	Verbatim code/text, or a tight summary with direct quotes of the important lines.
Accepted	What you kept, and why .
Rejected / modified	What you threw away or changed, and why . Be specific: bug, style issue, wrong abstraction, hallucinated API, etc.
Errors you caught	Any mistakes the AI made. How did you detect them? What would have happened if you hadn't?
Reflection sentences) (2–3	What did you learn about the problem — or about how to use AI — from this session?

Worked example entry (for reference)

Sample entry

Entry # 4 of 12 • **Date** 2026-04-02 • **Phase** Parser • **AI tool** Claude Opus 4.7

Goal. Write the recursive-descent parser for the expression grammar.

Prompt. “Here is my EBNF grammar for expressions [pasted]. Write a recursive-descent parser in Python that returns an AST with classes BinOp, UnaryOp, NumLit, Ident. No parser combinator libraries.”

Accepted. The overall structure (one function per non-terminal) and the BinOp(op, left, right) AST shape — matches my design.

Rejected / modified. The AI made parseExpr right-associative by recursing on itself before consuming the operator. My grammar specifies **left**-associativity. I rewrote it as a loop that consumes operators in sequence.

Errors caught. (1) Right-associativity bug above. (2) It omitted handling of unary minus entirely. Caught by running on the test input $-3 + 2$, which parsed as $-(3 + 2)$.

Reflection. AI is good at boilerplate parser structure but unreliable on precedence/associativity decisions — exactly the parts that encode the language designer's intent. Next time I'll include a precedence table in the prompt and check associativity with a dedicated test before accepting anything.

9. Academic Integrity

- AI assistance is **required and encouraged**, but must be **honestly documented**.
- Copying another student's (or another pair's) design, code, or rationale is plagiarism.

- Using AI to generate a design rationale that you cannot defend in the written exam is treated as a failure of the exam and will propagate to the written-artifact grade.
- **Within a pair:** the contribution report (D6) must be honest. Each partner's exam questions will target the components they personally claimed to have written. A partner who claims work they did not do will be unable to answer those questions and will be graded down individually.
- Two submissions (from different students or pairs) whose designs are suspiciously similar will both be investigated. Make original choices — that is the whole point of the project.
- If in doubt, ask the instructor before submitting.

Final note

This project is designed to be **genuinely hard to game** and **genuinely educational**. If you treat it as an AI-copy-paste exercise, you will struggle in the exams. If you treat it as what it is — a chance to make real programming-language design decisions and defend them — you will come out of it understanding every chapter of the book in a way no traditional exam can teach. Good luck.